

UML — Unified Modeling Language

O que é

UML (Unified Modeling Language, ou Linguagem de Modelagem Unificada) é uma linguagem visual padronizada para especificar, visualizar, construir e documentar os artefatos de um sistema de software. Não é uma linguagem de programação: não executa, não compila — serve para representar graficamente a estrutura e o comportamento de um sistema antes, durante e depois da implementação.

Mantida pela OMG (Object Management Group), surgiu em 1997 da unificação de três notações concorrentes (Booch, OMT de Rumbaugh, e OOSE de Jacobson), os chamados “três amigos” da modelagem orientada a objetos.

Para que serve

- **Comunicação:** uma notação comum entre desenvolvedores, arquitetos, stakeholders e professores/orientadores — todos leem o mesmo desenho.
- **Documentação:** registra decisões de projeto (estrutura de classes, fluxo de casos de uso, arquitetura de implantação) de forma durável, independente do código-fonte.
- **Análise e projeto:** ajuda a pensar a solução antes de codificar — identificar entidades, responsabilidades, relacionamentos e fluxos.
- **Rastreabilidade:** liga requisitos a elementos de projeto, facilitando auditoria e manutenção.

As duas grandes famílias de diagramas

UML define 14 tipos de diagrama, divididos em dois grupos.

1. Diagramas Estruturais (o que o sistema é)

Mostram a organização estática dos elementos.

Diagrama	Representa
Classes	Classes, atributos, métodos e relacionamentos (associação, herança, agregação, composição)
Objetos	Instâncias concretas de classes em um momento específico
Componentes	Módulos/componentes de software e suas interfaces
Implantação (Deployment)	Distribuição física do software em nós de hardware/infraestrutura
Pacotes	Agrupamento e dependência entre pacotes/namespaces
Estrutura Composta	Estrutura interna de uma classe/componente e suas colaborações
Perfil	Extensões e estereótipos customizados da própria UML

2. Diagramas Comportamentais (o que o sistema *faz*)

Mostram interação, fluxo e mudança de estado ao longo do tempo.

Diagrama	Representa
Casos de Uso	Funcionalidades do sistema sob a ótica de atores externos
Sequência	Ordem temporal de troca de mensagens entre objetos
Atividades	Fluxo de controle/processo, semelhante a um fluxograma
Máquina de Estados	Estados possíveis de um objeto e as transições entre eles
Comunicação	Interações entre objetos com foco na topologia (quem fala com quem)
Visão Geral de Interação	Combina diagrama de atividades com diagramas de sequência
Tempo (Timing)	Comportamento de objetos ao longo de uma linha do tempo

Elementos e notações essenciais

- **Classe:** retângulo dividido em três compartimentos — nome, atributos, métodos. Visibilidade indicada por + (público), - (privado), # (protegido), ~ (pacote).
- **Relacionamentos:**
 - *Associação* (linha simples) — uma classe usa/conhece a outra.
 - *Agregação* (losango vazio) — relação “todo-parte” fraca; a parte existe independente do todo.
 - *Composição* (losango cheio) — relação “todo-parte” forte; a parte não existe sem o todo.
 - *Herança/Generalização* (seta com ponta vazia) — especialização de uma classe base.
 - *Realização/Implementação* (seta tracejada com ponta vazia) — classe implementa uma interface.
 - *Dependência* (seta tracejada) — uma classe depende de outra, sem associação estrutural fixa.
- **Multiplicidade:** indica quantas instâncias participam da relação (1, 0..1, 1..*, 0..*, etc.).
- **Ator:** boneco (*stick figure*) em diagramas de caso de uso, representa um usuário ou sistema externo que interage com o sistema.
- **Nó (node):** cubo em diagramas de implantação, representa um recurso computacional físico ou virtual (servidor, dispositivo, container).

Por que usar em um TCC / projeto de software

- Formaliza requisitos funcionais em **casos de uso**.
- Formaliza a arquitetura estática em **diagrama de classes**.
- Formaliza a arquitetura física/implantação em **diagrama de implantação**.
- Dá rastreabilidade entre requisito → classe/método → execução, essencial para bancas e revisões técnicas.
- É a notação mais reconhecida academicamente para esse tipo de documentação, o que facilita avaliação por orientadores.

Ferramentas comuns

PlantUML, Mermaid, draw.io/diagrams.net, StarUML, Visual Paradigm, Enterprise Architect — todas geram os mesmos diagramas UML a partir de texto (PlantUML/Mermaid) ou de edição visual (draw.io, StarUML).

Referência

OMG UML Specification — <https://www.omg.org/spec/UML/>